

Dynamic memory allocation

C programming · CM10 · 90 minutes

ISTIC · University of Rennes 2026-2027



Louis Ledoux

\$ whoami

Not a C expert ... : ')

Louis Ledoux

Associate Professor

University of Rennes · ISTIC

IRISA / Inria · Taran

► Arithmetic circuits: **MLIR (LLVM)**
/ **CIRCT**.

► **Systolic arrays** and small
floating-point formats.

Previously: Inria / INSA Lyon,
Emeraude.

PhD: **BSC** / **UPC**.

bynaryman.github.io



MareNostrum 4 · BSC,
Barcelona

Dynamic memory allocation on the heap

We open an image and discover its dimensions. Where do we store its pixels?

1. Make an independent copy.
2. Store values and find memory errors.
3. Store a grid of values.
4. Group data with user-defined types.

1. Copy a string

Can we change the copy and keep the original?

A copy we can edit

Keep **hello** in the original.

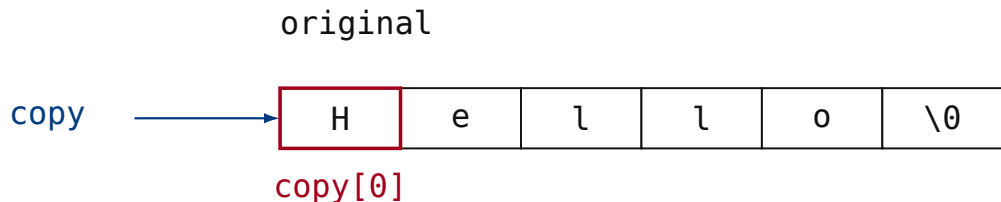
Change its copy to **Hello**.

What will the two lines print?

```
char original[] = "hello";  
char *copy = original;  
  
copy[0] = 'H';  
printf("original: %s\n", original);  
printf("copy:      %s\n", copy);
```

original: Hello
copy: Hello

One array, two ways to reach it



`char *copy = original;` copies the **address**.

Separate storage

How many bytes does the copy need?

```
char *copy = malloc(strlen(original));  
if (copy == NULL)  
    return EXIT_FAILURE;
```

malloc requests bytes. Check its result **before using it**.

Copying the characters

What should the two lines print now?

```
strcpy(copy, original);  
copy[0] = 'H';  
printf("original: %s\n", original);  
printf("copy:      %s\n", copy);
```

strcpy copies the characters **and the final '\0'**.

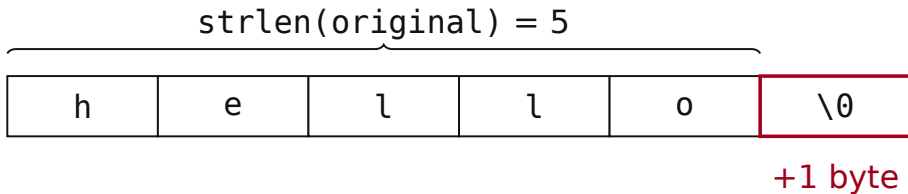
It prints. Is the copy valid?

```
make valgrind FILE=live-copy.c
```

Invalid write in strcpy

The destination block has **5 bytes**.

Five letters, six bytes



```
char *copy = malloc(strlen(original) + 1);
```

The copy works. What remains?

Read the report from the corrected allocation.

Which issue remains?

6 bytes definitely lost

We allocated the copy but never released it.

Releasing the copy

Where should `free(copy)` go?

```
printf("%s\n", original);  
printf("%s\n", copy);  
free(copy);
```

Release the block **after its last use**.

No leak: Valgrind reports zero errors.

Two independent strings

original

h	e	l	l	o	\0
---	---	---	---	---	----

copy



H	e	l	l	o	\0
---	---	---	---	---	----

separate allocation

Before free: changing `copy[0]` leaves `original[0]` unchanged.

2. Arrays and memory errors

We copied existing characters. Now we create an array of values.
How many **bytes**? What values are there **before we write**?

How much space for integers?

We need space for **100 integers**, initially set to zero.
How many bytes should we request?

Count elements, request bytes

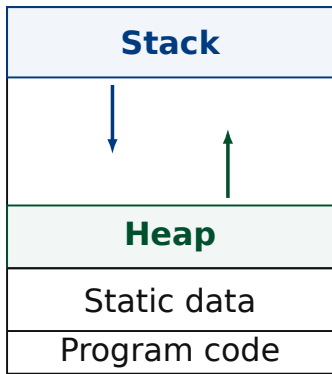
```
void *malloc(size_t size);    // From <stdlib.h>  
int *p = malloc(100 * sizeof(int));
```

- ▶ `size_t` is unsigned, with at least **16 value bits**.
- ▶ `sizeof` is an **operator** giving a size in bytes.
- ▶ `sizeof(int)` is the size of one `int`, not the pointer.

Sizes depend on the platform. In C, the `void *` result needs no cast.

Where does the program keep its data?

0xffff



count and pointer p
Automatic locals

Room to grow
Opposing directions

p[0] to p[count-1]
Allocated integers

Globals and static

0x0000

What happens if we remove the loop that writes zeros?

What will the array contain?

malloc succeeded. We removed the loop that wrote zeros.

```
printf("first=%d, last=%d\n",  
      p[0], p[count - 1]);
```

Uninitialised value

Valgrind reports that
printf uses values we
never wrote.

malloc reserves storage; it **does not initialise the integers**.

Write before reading

Restore the loop **before** printf.

```
for (int i = 0; i < count; ++i)
    p[i] = 0;

printf("first=%d, last=%d\n",
      p[0], p[count - 1]);
```

Every element now has a value before it is read.

Zero errors
Rerun Valgrind to
check this execution.

Four integers: which write is outside?

`count = 4`. The allocation holds `p[0]` through `p[3]`.

```
for (int i = 0; i <= count; ++i)
    p[i] = 0;
```

Invalid write

`<=` also allows `i = 4`.

There is **no element** `p[4]`
in this allocation.

Use `i < count`: four writes, with indices **0, 1, 2, 3**.

Can we read after free?

p points to four integers initialised to zero.

```
free(p);  
  
printf("first=%d, last=%d\n",  
      p[0], p[count - 1]);
```

After free(p), **p is a dangling pointer.**

Invalid read

Valgrind points to printf:
the block was **already
freed.**

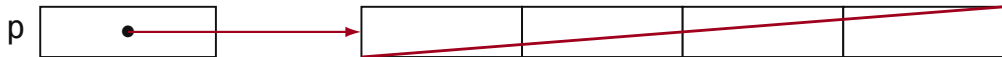
Dangling pointer

A pointer to an object **whose lifetime has ended**.

`free(p)` releases the block; it **does not set p to NULL**.

Pointer

Released block



Do not access these elements

Repair: finish reading before releasing the block.

```
printf("first=%d, last=%d\n", p[0], p[count - 1]);  
free(p);
```

Can we return this array?

Move array creation into a function. Can its caller use the array?

```
int *make_values(void) {  
    int values[4] = {0};  
    return values;  
}
```

```
// In the caller:  
int *p = make_values();
```

During the call
values exists and
contains four zeros.

After the return
values no longer exists. **p**
is dangling, without any
free.

Returning an address **does not keep a local object alive.**

When does the storage end?

The pointer and the object it refers to have **separate lifetimes**.

Storage	Lifetime ends
Ordinary local array	When its block ends
Block from malloc	When it is released with free
Global or static object	When the program ends

Each recursive call has its own locals. Too many calls can **exhaust the stack**.

Common mistake: lost address

We still call `free(p)`. **What value does it receive?**

```
p = NULL;  
free(p);
```

`free(NULL)` does nothing.
Address lost: **memory leak**.

Free before resetting.

```
free(p);  
p = NULL;
```

Pointer

p	NULL
---	------

Allocated block

0	0	0	0
---	---	---	---

Address lost; block still allocated

Who releases an unreachable block?

C: explicit responsibility

For every successful allocation, decide who will release it and when.
Repeated leaks consume more memory.

Java: garbage collection

The runtime can reclaim unreachable objects automatically.

Four functions in `<stdlib.h>`

We allocated, filled and released a block. We can also request zeros or resize it.

Function	Purpose
<code>malloc(bytes)</code>	Allocate uninitialised storage
<code>calloc(count, size)</code>	Allocate storage with all bits zero
<code>realloc(p, bytes)</code>	Resize an allocation; it may move
<code>free(p)</code>	Release an allocation

The free contract

```
void free(void *pointer);    // Declared in <stdlib.h>
```

- ▶ Pass the allocation's **start address**, once.
- ▶ `free(NULL)` does nothing; `free` returns no value.
- ▶ An invalid or repeated release is **undefined behaviour**.

Checkpoint · questions 1-5

Answer questions **1-5**, then discuss your reasoning.
Open the allocation quiz.

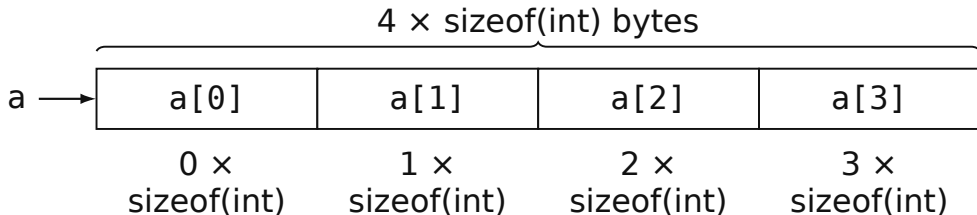
3. Store a grid

We know how to allocate **one row**. How can we store a **whole grid**?

One row in contiguous memory

What changes between neighbouring element addresses?

```
int *a = malloc(4 * sizeof(int));
```

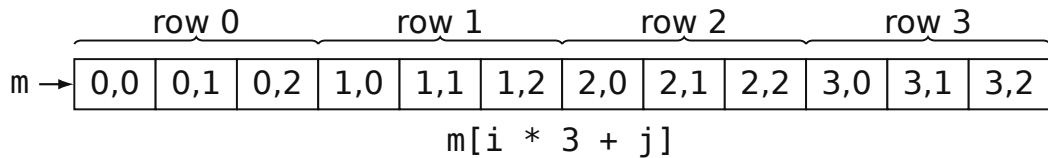


Use `a[i]`, equivalently `*(a + i)`, for **$0 \leq i < 4$** .

A 2D array in one block

We need **4 rows of 3 integers**. Which index stores row **2**, column **1**?

```
int *m = malloc(4 * 3 * sizeof(int));
```



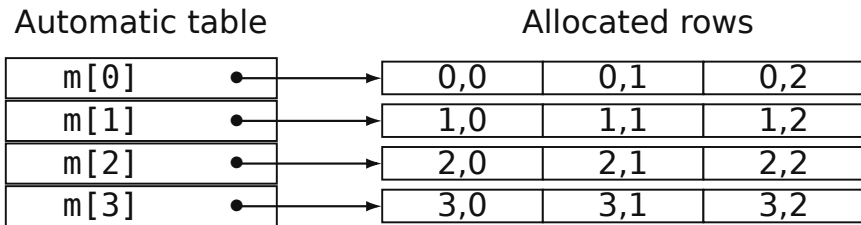
$m[i * 3 + j]$: row 2, column 1 is index **7**.

One allocation, one **free(m)**.

An array of row pointers

Which objects need free: the pointer table, the rows, or both?

```
int *m[4]; // Automatic table of four pointers.  
for (int i = 0; i < 4; ++i)  
    m[i] = malloc(3 * sizeof(int));
```

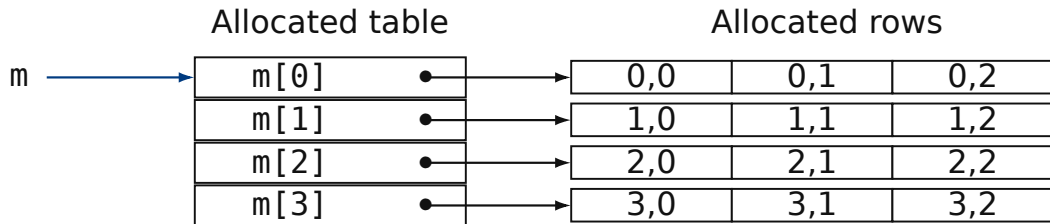


Check each row. Use `m[i][j]`; free each row, **not** `m`.

Allocate the pointer table too

The pointer table is now allocated too. How many blocks must we release?

```
int **m = malloc(4 * sizeof(int *));  
for (int i = 0; i < 4; ++i)  
    m[i] = malloc(3 * sizeof(int));
```



Check `m` first, then each row. Free **rows first**, then `free(m)`.

Checkpoint · questions 6-8

Answer questions **6-8**, then discuss your reasoning.
Open the allocation quiz.

4. User-defined types

An array groups values of one type. How can one object hold a street name **and** a house number?

User-defined types

One address contains a street name and a house number.

- ▶ struct: group the street and number.
- ▶ enum: name integer constants.
- ▶ typedef: give a type another name.

What integer value will today hold?

```
enum day { mon, tue, wed, thu, fri, sat, sun };  
enum day today;  
today = wed;
```

By default, values start at **0** and increase by **1**.

Named integer constants

mon is 0; wed is 2; sun is 6.

A structure groups several members

Does this create an address object?

```
struct address {  
    char street[100];  
    int number;  
};
```

A type definition

Members may be integers, floating-point values, arrays, pointers or other structures. No address object exists yet.

Define a type, then create objects

How many objects exist? Are their members ready to read?

```
struct address home;  
struct address office;
```

These definitions allocate **two distinct objects** of the same type.

Local objects need initialisation

Ordinary local members have indeterminate values until initialised or assigned. The type definition alone sets no values.

Initialising a structure

Which value goes into each member?

```
struct address home = { "Paul Bert", 12 };
```

Or combine the type and object definitions:

```
struct address {  
    char street[100];  
    int number;  
} home = { "Paul Bert", 12 };
```

Access members through an object

What will printing home show after these assignments?

```
struct address home = { "Paul Bert", 12 };  
strcpy(home.street, "Rue de Paris");  
home.number = 14;
```

The dot operator

Use `.` with a structure object.

`strcpy` needs `<string.h>` and enough space for the terminator.

Access members through a pointer

Does changing through `p` also change `home`?

```
struct address *p = &home;  
strcpy(p->street, "Rue de Paris");  
p->number = 16;
```

The arrow operator

Use `->` with a pointer. `p->number` is the same member as `(*p).number`.

Both forms above modify **home**.

Same members, different order

Same member types. Must these structures have the same size?

```
struct grouped {  
    int id1, id2;  
    char c1, c2;  
    float ratio;  
};  
struct interleaved {  
    int id1; char c1;  
    int id2; char c2;  
    float ratio;  
};
```

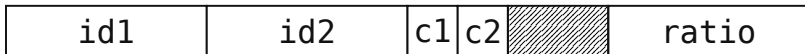
Padding

The implementation may insert unused bytes between members and at the end to satisfy alignment requirements.

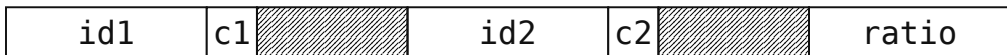
- ▶ Members appear in declaration order.
- ▶ `sizeof(struct T)` includes padding.
- ▶ Layout and alignment depend on the implementation.

See where the padding goes

Grouped: 16 bytes



Interleaved: 20 bytes



Here: 4-byte int/float, alignment 4; 1-byte char.

16 vs 20 bytes in this layout. Measure with `sizeof` and `offsetof`.

typedef: a name for a type

Does naming a type create an object?

```
typedef int LENGTH;  
typedef enum { FALSE, TRUE } BOOLEAN;  
typedef struct date DATE;
```

Alias

A typedef introduces a type name, not an object or a distinct underlying type. Then write `LENGTH width;`.

Combine a date and a day of the week

```
struct date { short day, month; int year; };  
typedef enum { mon, tue, wed, thu, fri, sat, sun } DAY;  
typedef struct { DAY day; struct date date; } A_DATE;
```

A nested structure

A_DATE contains a DAY and a complete struct date.

Access nested members

What year will `today.date.year` contain after the update through `p`?

```
A_DATE today = { wed, { 3, 9, 2014 } };  
today.day = wed;  
today.date.year = 2014;  
A_DATE *p = &today;  
p->day = today.day;  
p->date.year = 2026;
```

`p->date.year` reaches the nested member of **the same object**.

Checkpoint · questions 9-12

Answer questions **9-12**, then discuss your reasoning.
Open the allocation quiz.